

MicroBlog

End-to-End DevOps Project

Step-by-Step Guide for Final Semester Students

[Git](#)[Docker](#)[Kubernetes](#)[Terraform](#)[CI/CD](#)[Demo](#)

Project Overview

MicroBlog is a 3-tier blogging web application that students will build and deploy using the complete DevOps toolchain. The application is intentionally simple — a minimal blog with posts — so that all effort goes into the deployment pipeline, not the business logic. Every tool you have studied in class is used in this project.

What You Are Building

Component	Technology
Frontend	HTML + Nginx (serves static pages)
Backend	Python Flask REST API
Database	PostgreSQL (containerised)
Containerisation	Docker + Docker Compose
Orchestration	Kubernetes via Minikube (local)
Infrastructure as Code	Terraform (local provider)
CI/CD Pipeline	GitHub Actions (free)
Source Control	Git + GitHub (public repo)

Tools to Install Before Starting

Tool	How to Get It
Docker Desktop	https://www.docker.com/products/docker-desktop
Minikube	https://minikube.sigs.k8s.io/docs/start/

kubectl	Included with Minikube: minikube kubectl
Terraform	https://developer.hashicorp.com/terraform/install
Git	https://git-scm.com/downloads
VS Code (recommended)	https://code.visualstudio.com

Repository Folder Structure

Create this exact structure in your GitHub repository:

```
microblog/  
  frontend/  
    index.html          <- The blog web page  
    nginx.conf          <- Nginx reverse proxy config  
    Dockerfile          <- Frontend container image  
  backend/  
    app.py              <- Flask REST API  
    requirements.txt     <- Python dependencies  
    Dockerfile          <- Backend container image  
  k8s/  
    postgres-deployment.yaml  
    postgres-service.yaml  
    backend-deployment.yaml  
    backend-service.yaml  
    frontend-deployment.yaml  
    frontend-service.yaml  
  terraform/  
    main.tf             <- Resource definitions  
    variables.tf        <- Input variables  
    outputs.tf          <- Output values  
  .github/  
    workflows/  
      ci.yml            <- GitHub Actions pipeline  
  docker-compose.yml    <- Run everything locally  
  README.md             <- Professional documentation
```

WEEK 1
Docker**Application + Docker + Docker Compose**

Goal

By the end of Week 1, your entire 3-tier application must run on any laptop with one command: `docker-compose up` — with zero other steps required.

Step 1 — Create the GitHub Repository

1. Go to github.com and create a new public repository named `microblog`
2. Clone it locally:

```
git clone https://github.com/YOUR_USERNAME/microblog.git
cd microblog
```

3. Create all folders:

```
mkdir -p frontend backend k8s terraform .github/workflows
```

Step 2 — Write the Backend API (backend/app.py)

```
from flask import Flask, jsonify, request
import psycopg2, os

app = Flask(__name__)

def get_db():
    return psycopg2.connect(
        host=os.environ.get('DB_HOST', 'db'),
        database=os.environ.get('DB_NAME', 'microblog'),
        user=os.environ.get('DB_USER', 'admin'),
        password=os.environ.get('DB_PASS', 'password123')
    )

@app.route('/posts', methods=['GET'])
def get_posts():
    conn = get_db()
    cur = conn.cursor()
    cur.execute('SELECT id, title, body FROM posts ORDER BY id DESC')
    rows = cur.fetchall()
    return jsonify([{'id':r[0], 'title':r[1], 'body':r[2]} for r in rows])

@app.route('/posts', methods=['POST'])
def create_post():
    data = request.json
    conn = get_db()
    cur = conn.cursor()
    cur.execute('INSERT INTO posts(title,body) VALUES(%s,%s)',
```

```
        (data['title'], data['body']))
    conn.commit()
    return jsonify({'message': 'Post created'}), 201

@app.route('/health')
def health(): return jsonify({'status': 'ok'})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Create backend/requirements.txt:

```
flask==3.0.0
psycopg2-binary==2.9.9
gunicorn==21.2.0
```

Step 3 — Write the Backend Dockerfile (backend/Dockerfile)

```
# Use official Python slim image
FROM python:3.11-slim
# Set working directory inside the container
WORKDIR /app
# Install dependencies first (Docker layer caching optimisation)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copy the application code
COPY . .
EXPOSE 5000
# Production server: gunicorn with 2 workers
CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:5000", "app:app"]
```

Step 4 — Write the Frontend (frontend/index.html)

```
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <title>MicroBlog</title>
  <style>
    body { font-family: Arial; max-width: 700px; margin: 40px auto; padding: 0 20px; }
    .post { border: 1px solid #ddd; padding: 16px; margin: 12px 0; border-radius: 6px; }
    input, textarea { width: 100%; padding: 8px; margin: 6px 0; box-sizing: border-box; }
  </style>
</head>
<body>
  <h1>MicroBlog</h1>
  <form id='postForm'>
    <input id='title' placeholder='Post title' required>
```

```
<textarea id='body' placeholder='Write your post...' rows='4' required></textarea>
<button type='submit'>Publish</button>
</form>
<div id='posts'></div>
<script>
  const API = '/api';
  async function loadPosts() {
    const res = await fetch(API + '/posts');
    const posts = await res.json();
    document.getElementById('posts').innerHTML =
      posts.map(p => `<div
class='post'><h3>${p.title}</h3><p>${p.body}</p></div>`).join('');
  }
  document.getElementById('postForm').onsubmit = async (e) => {
    e.preventDefault();
    await fetch(API + '/posts', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        title: document.getElementById('title').value,
        body: document.getElementById('body').value
      })
    });
    loadPosts();
  };
  loadPosts();
</script>
</body></html>
```

Create frontend/nginx.conf (reverse proxy routes /api to backend):

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;
  location /api/ {
    proxy_pass http://backend:5000/;
  }
}
```

Create frontend/Dockerfile:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Step 5 — Write docker-compose.yml (root folder)

```
version: '3.8'
```

```
services:

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: microblog
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password123
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "admin"]
      interval: 5s
      retries: 5

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: microblog
      DB_USER: admin
      DB_PASS: password123
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - backend

volumes:
  postgres_data:
```

Step 6 — Run, Test, and Push

4. Build and start all services:

```
docker-compose up --build
```

5. Open browser at <http://localhost:8080> and create a test post

6. Test the API directly:

```
curl http://localhost:5000/health
curl http://localhost:5000/posts
```

7. Push your image to Docker Hub:

```
docker login
docker tag microblog-backend YOUR_USERNAME/microblog-backend:v1
```

```
docker tag microblog-frontend YOUR_USERNAME/microblog-frontend:v1
docker push YOUR_USERNAME/microblog-backend:v1
docker push YOUR_USERNAME/microblog-frontend:v1
```

8. Commit everything to GitHub:

```
git add .
git commit -m "Week 1: Dockerized 3-tier MicroBlog app"
git push origin main
```

Week 1 Deliverable Run docker-compose up on a teammate's laptop from a fresh clone. The app must work with zero extra steps.

WEEK 2
Kubernetes**Kubernetes with Minikube**

Goal

Deploy the same app on a local Kubernetes cluster using Minikube. Show 2 replicas, rolling updates, and self-healing — these are the 3 things interviewers ask about most.

Step 1 — Start Minikube

9. Start the local cluster:

```
minikube start --driver=docker
```

10. Verify the node is ready:

```
kubectl get nodes
```

Expected output: one node with STATUS = Ready

11. Enable ingress:

```
minikube addons enable ingress
```

Step 2 — Create Kubernetes Manifests

Create k8s/postgres-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          env:
            - name: POSTGRES_DB
              value: microblog
            - name: POSTGRES_USER
              value: admin
            - name: POSTGRES_PASSWORD
              value: password123
```



```
ports:
  - containerPort: 5432
```

Create k8s/postgres-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

Create k8s/backend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: YOUR_USERNAME/microblog-backend:v1
          ports:
            - containerPort: 5000
          env:
            - name: DB_HOST
              value: db
            - name: DB_NAME
              value: microblog
            - name: DB_USER
              value: admin
            - name: DB_PASS
              value: password123
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 10
```

Create k8s/backend-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  type: NodePort
  selector:
    app: backend
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30500
```

Create k8s/frontend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: YOUR_USERNAME/microblog-frontend:v1
          ports:
            - containerPort: 80
```

Create k8s/frontend-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Step 3 — Deploy Everything

12. Apply all manifests at once:

```
kubectl apply -f k8s/
```

13. Watch pods start (takes 1-2 minutes):

```
kubectl get pods --watch
```

14. When all pods show Running, open the app:

```
minikube service frontend
```

Step 4 — Demonstrate Key Features

Rolling Update — deploy a new version with zero downtime:

```
kubectl set image deployment/backend backend=YOUR_USERNAME/microblog-backend:v2  
kubectl rollout status deployment/backend
```

Self-Healing — delete a pod and watch Kubernetes restart it:

```
kubectl delete pod -l app=backend  
kubectl get pods --watch
```

Scaling — increase replicas on demand:

```
kubectl scale deployment backend --replicas=4  
kubectl get pods
```

Week 2 Deliverable Show kubectl get pods with 2 backend replicas both Running. Delete one pod live and show Kubernetes auto-recovering it within seconds.

WEEK 3
Terraform**Terraform — Infrastructure as Code**

Goal

Learn Terraform's core workflow (init → plan → apply) using the local provider. No cloud account is needed. Understand variables, outputs, and the state file concept.

Step 1 — Create terraform/variables.tf

```
variable "project_name" {
  description = "Name of the project"
  default     = "microblog"
}

variable "environment" {
  description = "Deployment environment"
  default     = "local"
}

variable "replicas" {
  description = "Number of backend replicas"
  type        = number
  default     = 2
}
```

Step 2 — Create terraform/main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

provider "local" {}

# Generate a Kubernetes ConfigMap from Terraform variables
resource "local_file" "k8s_config" {
  filename = "${path.module}/../k8s/generated-config.yaml"
  content  = <<-EOT
    # Generated by Terraform - DO NOT EDIT MANUALLY
    apiVersion: v1
    kind: ConfigMap
    metadata:
```

```
    name: app-config
  data:
    PROJECT_NAME: "${var.project_name}"
    ENVIRONMENT:  "${var.environment}"
    REPLICAS:     "${var.replicas}"
  EOT
}

# Write a human-readable deployment summary
resource "local_file" "summary" {
  filename = "${path.module}/../deployment-summary.txt"
  content  = <<-EOT
    Project:    ${var.project_name}
    Environment: ${var.environment}
    Replicas:   ${var.replicas}
  EOT
}
```

Step 3 — Create terraform/outputs.tf

```
output "project_name" {
  value = var.project_name
}

output "config_file" {
  value = local_file.k8s_config.filename
}

output "environment" {
  value = var.environment
}
```

Step 4 — Run the Full Terraform Workflow

15. Navigate to the terraform directory:

```
cd terraform
```

16. Initialise — downloads the local provider:

```
terraform init
```

Expected: Terraform has been successfully initialized!

17. Plan — preview what will be created (no changes yet):

```
terraform plan
```

18. Apply — create the files:

```
terraform apply
```

Type yes when prompted.

19. View outputs:

```
terraform output
```

20. Change a variable without editing the file:

```
terraform apply -var="replicas=3"
```

21. Inspect the state file (how Terraform tracks what it manages):

```
cat terraform.tfstate
```

22. Clean up:

```
terraform destroy
```

Week 3 Deliverable Show terraform plan output in your demo. Add .tf files to GitHub but add terraform.tfstate to .gitignore — never commit state files.

WEEK 4
CI/CD**CI/CD Pipeline with GitHub Actions**

Goal

Automate everything: one git push triggers code linting, Docker build, push to Docker Hub, and a Kubernetes deployment update. No manual steps.

Step 1 — Add GitHub Secrets

Go to: GitHub repo > Settings > Secrets and variables > Actions > New repository secret

Secret Name	Value
DOCKER_USERNAME	Your Docker Hub username
DOCKER_PASSWORD	Your Docker Hub password or access token

Step 2 — Write .github/workflows/ci.yml

```
name: MicroBlog CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:

  lint:
    name: Lint Python Code
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install flake8
        run: pip install flake8
      - name: Lint
        run: flake8 backend/app.py --max-line-length=100

  build-and-push:
```

```
steps:
  - uses: actions/checkout@v4

  - name: Log in to Docker Hub
    uses: docker/login-action@v3
    with:
      username: ${ secrets.DOCKER_USERNAME }
      password: ${ secrets.DOCKER_PASSWORD }

  - name: Build and push backend
    uses: docker/build-push-action@v5
    with:
      context: ./backend
      push: true
      tags: |
        ${ secrets.DOCKER_USERNAME }/microblog-backend:latest
        ${ secrets.DOCKER_USERNAME }/microblog-backend:${ github.sha }

  - name: Build and push frontend
    uses: docker/build-push-action@v5
    with:
      context: ./frontend
      push: true
      tags: |
        ${ secrets.DOCKER_USERNAME }/microblog-frontend:latest
        ${ secrets.DOCKER_USERNAME }/microblog-frontend:${ github.sha }

notify:
  name: Pipeline Summary
  needs: build-and-push
  runs-on: ubuntu-latest
  steps:
    - run: |
        echo '===== '
        echo 'MicroBlog pipeline complete!'
        echo 'Commit: ${ github.sha }'
        echo 'Images pushed to Docker Hub'
        echo '===== '
```

Step 3 — Trigger and Verify the Pipeline

23. Make any change (e.g. add a comment to app.py)

24. Commit and push:

```
git add .
git commit -m "Week 4: Add CI/CD pipeline with GitHub Actions"
git push origin main
```

25. Open GitHub > Actions tab — watch the pipeline run live

26. After the green checkmark, verify new images on hub.docker.com

27. Update Kubernetes to use the new image:


```
kubectl set image deployment/backend backend=YOUR_USERNAME/microblog-backend:latest  
kubectl rollout status deployment/backend
```

Week 4 Deliverable Show the GitHub Actions tab with a green checkmark for all 3 jobs: lint, build-and-push, and notify.

WEEK 5
Demo**Demo, Documentation & CV Preparation**

Goal

Deliver a polished live demo, write a professional README that impresses recruiters, and craft a CV bullet that immediately signals real-world DevOps experience.

Step 1 — Write README.md

Your README is the first thing a recruiter sees when they open your GitHub link. It must include:

- Project title and a one-line description
- Architecture diagram — even ASCII is fine
- Tech stack listed clearly
- Prerequisites section
- Quick start — 3 commands maximum to run the app
- Screenshot of the working app in browser
- Screenshot of GitHub Actions green pipeline
- Author names and student IDs

Minimum README template:

```
# MicroBlog
A 3-tier blogging app with a full DevOps pipeline.

## Architecture
Browser → Nginx (Frontend) → Flask (Backend) → PostgreSQL

## Stack
Docker | Kubernetes (Minikube) | Terraform | GitHub Actions

## Quick Start
git clone https://github.com/YOUR_USERNAME/microblog
cd microblog
docker-compose up --build
# Visit http://localhost:8080

## Kubernetes Deploy
minikube start
kubectl apply -f k8s/
minikube service frontend
```

Step 2 — Live Demo Order

Demo Step	What to Show
1. Docker Compose	docker-compose up, open browser, create a post
2. Kubernetes pods	kubectl get pods — 2 replicas for backend
3. Self-healing	Delete a pod live, watch it auto-restart
4. Rolling update	kubectl set image, kubectl rollout status
5. Terraform plan	terraform plan — show IaC in action
6. CI/CD pipeline	GitHub Actions tab — green checkmark
7. Docker Hub	Show tagged images on hub.docker.com
8. GitHub repo	Professional README with all files present

Step 3 — Your CV Bullet (Copy This)

MicroBlog DevOps Pipeline | github.com/YOUR_USERNAME/microblog

- Containerised a 3-tier Python/Nginx/PostgreSQL app using Docker and Docker Compose
- Orchestrated deployment on Kubernetes (Minikube) with 2 replicas, rolling updates, and self-healing
- Automated infrastructure provisioning using Terraform (Infrastructure as Code)
- Built CI/CD pipeline with GitHub Actions — auto-builds and pushes Docker images on every git push

Grading Rubric

Area	Marks
Docker — image builds, app runs via docker-compose up	20 marks
Kubernetes — 2 replicas, rolling update demonstrated live	25 marks
Terraform — plan output shown, .tf files committed to GitHub	20 marks
CI/CD — GitHub Actions green, auto-triggers on git push	25 marks
README — architecture,	10 marks

screenshots, clear
quick-start guide

Bonus —
Prometheus/Grafana
monitoring OR HTTPS
added

+10 marks

Golden Rule The GitHub repo IS your portfolio. Use clean commit messages like 'Week 2: Add Kubernetes manifests with 2 replicas'. Keep the repo public permanently. This is the link you share in every job application.